

CS633 Assignment 2 Report

Group 74

Rishab K S (Leader) 251110061
Kanchi Kandadai Bharath 251110042
Hemant Jagtap 251110037
Abhishek Dixit 251110003
Ashwin Kansagara 251110019

11-04-2026

1 Code Description

We wrote an MPI-based 3D stencil simulation that runs over T time steps. The domain is split across $px \times py \times pz$ processes, with each process holding an $nx \times ny \times nz$ chunk for F fields. Each time step does three things in order: halo exchange, stencil update, and isovalue crossing count. Inputs are $d, ppn, px, py, pz, nx, ny, nz, T, \text{seed}, F$, and isovalue; rank 0 prints F counts per step followed by the total elapsed time.

Domain Layout and Indexing

Each process computes its 3D position as $cx = \text{rank} \bmod px$, $cy = (\text{rank}/px) \bmod py$, $cz = \text{rank}/(px \cdot py)$. A halo of width $k = (d - 1)/6$ is added on all six faces, making the local buffer $(nx+2k) \times (ny+2k) \times (nz+2k)$. A small inline macro `IDX(iz,iy,ix)` handles the 3D-to-1D mapping. The global origin for a process is $(cx \cdot nx, cy \cdot ny, cz \cdot nz)$ and the full domain is $px \cdot nx \times py \cdot ny \times pz \cdot nz$. `get_rank(cx,cy,cz)` returns the MPI rank for a given grid coordinate, or -1 if it falls outside the process grid — checked before every send/receive.

Halo Exchange

Halos are exchanged one axis at a time: X first, then Y, then Z. For each axis we pack the interior k -thick face into a send buffer, post non-blocking `MPI_Irecv` and `MPI_Isend` calls to both neighbours, then wait with `MPI_Waitall` before unpacking into the ghost layer.

Doing the axes in order matters: when packing Y faces the X ghosts are already filled in, and by the time we pack Z faces both X and Y are done. This handles corners and edges correctly without any extra diagonal communication. Buffer sizes are:

$$xfs = k \cdot ny \cdot nz, \quad yfs = NX \cdot k \cdot nz, \quad zfs = NX \cdot NY \cdot k$$

The Y and Z buffers use the extended NX/NY so corner and edge ghost cells are included in those exchanges.

Stencil Update

For every interior point we sum contributions from neighbours up to distance k in each of the six axis directions. A neighbour is only added if its global coordinate falls inside the domain, so boundary points divide by the actual available count m rather than a fixed d . Results go into a separate `nxt` buffer; after each step we swap the `cur` and `nxt` pointers instead of copying data.

Isovalue Crossing Count

For each unit cube we read all 8 corners. If at least one corner is strictly below the isovalue and at least one is at or above it, the cube straddles the isovalue and is counted. This check implicitly covers face, edge, and body diagonal neighbours through the 8-corner cube approach and is consistent throughout. Cubes at inter-process boundaries are counted by both neighbouring

processes (double counting), while cubes at the global domain edge are excluded since no valid ghost exists there. Local counts are summed across all ranks with `MPI_Reduce` and printed by rank 0 after every time step.

2 Code Optimisation

What was slow: allocating buffers inside the exchange

The original version called `malloc` for all twelve send/receive buffers (`sxm`, `sxp`, `rxm`, `rxp`, `sym`, `syp`, `rym`, `ryp`, `szm`, `szp`, `rzx`, `rzp`) at the start of every `halo_exchange` call and `free`'d them on exit. Since `halo_exchange` runs $T \times F$ times, that meant $T \times F \times 12$ heap operations per process for buffers whose sizes never change between calls.

Fix: allocate once in `main()`

We moved all twelve allocations into `main()` before the time-step loop and passed the pointers into `halo_exchange` as arguments. The function just reuses them every call. For $T = 5$, $F = 2$ this removes $5 \times 2 \times 12 = 120$ `malloc/free` pairs per process, replacing them with one allocation each at startup.

How much it helped

Table 3 shows the median times. For $nz = 240$ the benefit is consistent across all process counts: $P = 32$ dropped from 12.49s to 11.54s ($\approx 7.6\%$), $P = 96$ from 13.75s to 12.96s ($\approx 5.7\%$). For $nz = 120$ the gains are smaller and less consistent because the per-step work is much lighter and the heap overhead is a smaller fraction of total runtime.

3 Compilation and Execution

Compiled on PARAM Rudra with:

```
mpicc src.c -o src -lm
```

A sample run at $P = 32$, sub-domain size 120:

```
mpirun -np 32 ./src 7 32 4 4 2 120 120 120 5 1000 2 500
```

Fixed parameters are in Table 1 and process grids in Table 2. We ran each of the 8 configurations 5 times and recorded the time from the last line of rank 0 output.

Table 1: Fixed parameters used across all configurations

Parameter	Value
d (stencil width)	7
T (time steps)	5
seed	1000
F (number of fields)	2
isovalue	500
$nx = ny = nz$	120 or 240

Table 2: Process grid configurations

P	px	py	pz	ppn
32	4	4	2	32
48	6	4	2	48
64	4	4	4	32
96	6	4	4	48

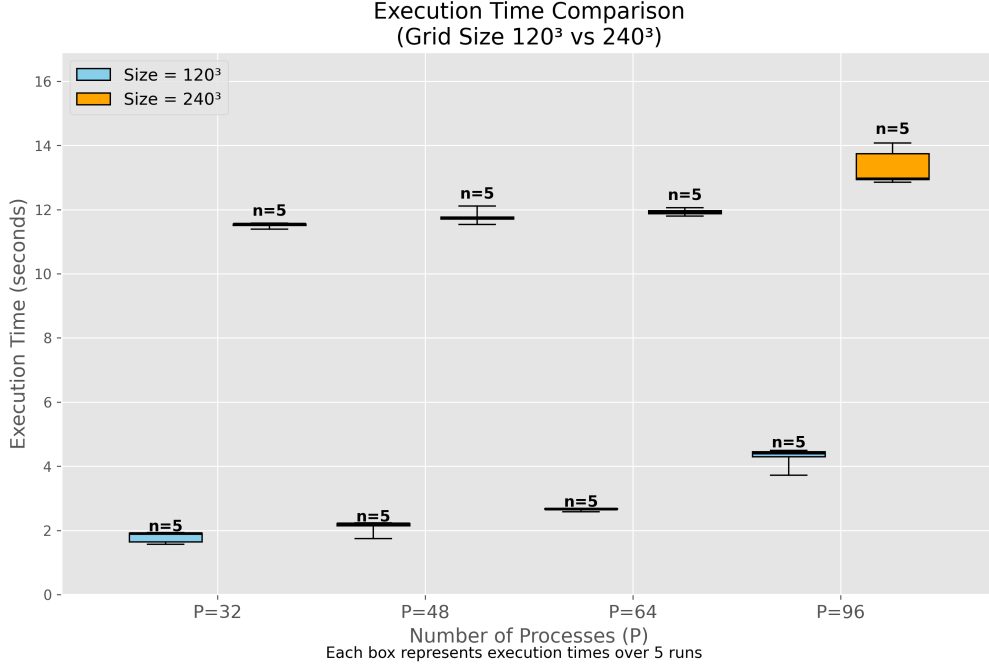


Figure 1: Execution time (seconds) for the optimised code across all configurations. Each box summarises 5 independent runs. Time is on the Y-axis and number of processes P on the X-axis.

4 Results and Observations

Figure 1 shows the timing distributions. Median runtimes are listed in Table 3.

Table 3: Median execution times (seconds) — optimised vs. unoptimised

P	Optimised		Unoptimised	
	$nz = 120$	$nz = 240$	$nz = 120$	$nz = 240$
32	1.90	11.54	2.06	12.49
48	2.20	11.73	1.95	12.84
64	2.67	11.94	2.74	12.92
96	4.41	12.96	4.38	13.75

Observations

- **Sub-domain size dominates runtime.** The $nz = 240$ cases are roughly 5–6 $\times$ slower than $nz = 120$ across every process count. Each process handles 8 $\times$ more grid points, and the Y and Z halo buffers grow with NX and NY , so communication volume also increases.
- **More processes makes things slower for this problem size.** For $nz = 120$, time rises from ≈ 1.90 s at $P = 32$ to ≈ 4.41 s at $P = 96$. For $nz = 240$ it goes from ≈ 11.54 s to ≈ 12.96 s. The per-process chunk shrinks as P grows, but the number of halo exchanges increases, and the communication overhead grows faster than the computation savings. The program is communication-bound at these scales.
- **$P = 48$ is slightly slower than $P = 32$ for $nz = 240$.** The median at $P = 32$ is 11.54 s versus 11.73 s at $P = 48$. Even though $P = 48$ uses a $6 \times 4 \times 2$ layout that reduces each process's nx and the X-face halo volume, having more ranks to communicate with outweighs this benefit.
- **The optimisation has mixed effect at $nz = 120$.** For $nz = 120$ the gains are small and in some cases (e.g. $P = 48$: optimised 2.20 s vs. unoptimised 1.95 s) the difference goes the other way. This is within run-to-run noise, the per-step computation is light enough that the

removed `malloc` overhead is not the dominant cost.

- **The buffer allocation fix gives a clear speedup at $nz = 240$.** At $P = 32$ the median fell from 12.49 s to 11.54 s ($\approx 7.6\%$). At $P = 96$ the gain was $\approx 5.7\%$, smaller because communication overhead is heavier there. The benefit scales with $T \times F$, so longer runs or more fields would show more improvement.
- **Run-to-run variation is generally small.** Most boxes in the plot are tight. A few outliers appear (e.g. $P = 96$, $nz = 240$, run 3 at 14.08 s for both optimised and unoptimised), likely from transient network or scheduler interference on the cluster.

5 Conclusion

The program handles arbitrary d -point stencils with F fields over T steps using only the MPI functions covered in class. Boundary points divide by the number of actually available neighbours, and the isovalue count uses a consistent 8-corner unit-cube crossing check throughout. Boundary cubes between processes are double-counted as required.

Moving the halo buffer allocations out of the inner loop was the main optimisation. It is a simple change but removes $T \times F \times 12$ repeated heap operations per process. The speedup is clearest at $nz = 240$: $\approx 7.6\%$ at $P = 32$ and $\approx 5.7\%$ at $P = 96$. At $nz = 120$ the effect is within noise because the per-step work is small relative to other overheads.

The main takeaway from the results is that the program is communication-bound at these scales. Going from $P = 32$ to $P = 96$ consistently increases runtime, which shows that the halo traffic is the bottleneck at the given problem sizes.